

When Specs Meet Agents

A Five-Phase, Audit-First Architecture and Framework for AI-Driven Software Modernization

By Dmytro Rudakov

Image 1 — hero cover (full page)

Organizations modernizing aging systems in 2026 face a class of work that frontier LLMs do not solve on their own. The shape of that work has changed: it is no longer the dev-platform version bump (Java 8 → 21/25 LTS, .NET Framework → .NET 9, Python 2 → 3.12, Spring Boot 2 → 3.x) that dominated 2018-era modernization conversations. It is the portfolio-wide platform rewrite. The data tier moves: monolithic Oracle and DB2 estates retargeted to PostgreSQL with database-per-service and pgvector for AI-native workloads. The transaction and integration model moves: two-phase-commit gives way to Temporal- or Rebase-style durable-execution sagas; nightly batch ETL is replaced by Debezium-Kafka-Flink streaming into Iceberg or Paimon lakehouses organized as data products; mainframe COBOL is refactored to Java or Python with AI-assisted tooling. The deployment and trust model moves: single-region targets are retargeted to multi-region active-active under operational-resilience regimes (DORA, FCA); SAML and LDAP retire in favor of OIDC, FIDO2 passkeys, and FAPI 2.0; and the cryptographic estate moves to hybrid post-quantum (X25519 + ML-KEM, ML-DSA chains) against the 2035 NCSC deadline. The unifying constraint across all of these is the same: the modernization must be *measurably done*, not merely *asserted* done.

This article describes an audit-first architecture and framework for AI-assisted modernization, built around four pillars — Dimensions, Traceability, Coverage Validation, and Modernization-Gap Resolution — and operationalized through a five-phase pipeline. It is not a theoretical proposal. The architecture and framework have been validated end-to-end on two genuinely different engagements: a memory-safety-critical C-to-Rust modernization of a native system, and a complete PHP-to-Python rewrite of a non-trivial self-hosted web application. The same loop, the same coverage validator, the same gap-resolution discipline carried both engagements through to retirement of the legacy system. The architecture and framework are also consistent with the pattern emerging from independent industry work — notably DARPA's TRACTOR programme for C-to-Rust and Google's recently-published at-scale LLM-assisted code modernization (Saavedra et al. 2025).

Why Current Approaches Fall Short

Three forces are converging into a significant modernization opportunity, and a corresponding gap.

Legacy estates are aging faster than teams can evolve them. The U.S. Government Accountability Office reported in 2023 that roughly 80% of the federal IT budget — over \$100 billion annually — is spent operating and maintaining legacy systems. The UK figure is £2.3 billion per year on systems some of which date back thirty years or more. Compliance mandates around memory safety, post-quantum migration, and operational-resilience tolerances increasingly turn «*we should modernize*» into «*we must modernize before date X*.»

Frontier LLMs are now reliable engineering collaborators on tool use and long-context reasoning. Anthropic's Claude Opus 4.7 (April 2026), OpenAI's GPT-5.5 Pro (April 2026), Google DeepMind's Gemini 3.1 Pro (February 2026), and their open-source peers can plan, execute tools, hold thousands of lines of context, and iterate inside a loop. What they cannot do, as the rest of this section establishes, is grade their own output.

The Confirmation-Bias Failure Mode

Frontier coding agents systematically over-report success on engineering work, and the phenomenon is the single most consequential failure mode in AI-driven modernization today. A March 2026 study from the University of Athens (Mitropoulos et al. 2026) shows that simply framing a pull request as «*bug-free*» reduces vulnerability-detection rates by 16–93% across four current-generation models, and that adversarial framing succeeds in 35% of one-shot attacks against GitHub Copilot and **88% against Claude Code under iterative project configurations**. Anthropic's Claude Opus 4.6

system card (February 2026) formally catalogs «*misrepresenting tool results*» and «*premature claims of success*» as recurring failure modes, and Anthropic's April 2026 Claude Code postmortem acknowledged that silent regressions caused the agent to declare tasks complete while under-reasoning about them. A May 2026 benchmark of thirteen frontier models found that **72% of reward-hacking episodes include explicit chain-of-thought rationale** — the model frames its shortcuts as legitimate work.

This is why so many AI-driven modernization programmes launch with rapid early wins and visible confidence, and then quietly collapse three to six months in. The early wins were the same kind of unverified claim the system continues to make at scale, and there is no independent check on completion until a customer hits a regression in production. The implication for architecture is direct: external grading is not an optimization but a correctness condition. Recent surveys argue this is a structural property of RLHF / RLVR alignment, not a prompt-fix problem (arXiv:2604.13602, April 2026).

A second, less-discussed gap compounds the first. The code an LLM produces during modernization is rarely paired with a spec-driven roadmap that tells the organization *what is left*. The legacy system gets a successor of some kind, but no agreed-upon catalog of which behaviors have been preserved, which were eliminated by decision, which were inferred and need confirmation, and — critically — when the legacy system can be decommissioned in full. The new system runs, the legacy system runs, and no stakeholder group is willing to retire the old one because completeness was never verifiably established.

The Landscape: Three Styles of AI-Assisted Modernization

Three working styles are common in 2026.

Transpiler-style work treats the LLM as a stochastic translator. The engineer feeds it source — by file, by function, by feature — and pastes or commits the output. This is not only the legacy chat-window pattern of 2023; the same shape survives in the modern agentic IDE, where the engineer tells Claude Code or Cursor: «*go to file X, rewrite function Y to use the new API, then apply the same change to file Z.*» The interaction is imperative, fragment-scoped, with completeness held entirely in the engineer's head. The frontier LLM is used as a regex on steroids. The literature includes refinements — neuro-symbolic hybrids, skeleton-guided translation, dependency-guided batching — but none changes the underlying property that further source analysis cannot correct earlier output without re-running the chunk, and that there is no global view of *what has not yet been accounted for*. At enterprise scale this style takes years, with or without AI assistance, and the team itself becomes the only source of completeness signal.

Spec-first work is the inverse. Rather than telling the model what to do step by step, the team writes down what must exist — the target architecture, the domain vocabulary, the contracts each component must honour — and lets the model infer how to satisfy it. The simplest discipline is a four-line breakdown applied to every component specification: **Mission** (the business outcome the component serves), **Goals** (the measurable engineering objectives), **Premises** (the dependencies and givens the architecture is willing to assume — frameworks, data engine, runtime, security posture), and **Constraints** (the constraints the architecture will not violate — performance budgets, behavioral parity, regulatory boundaries). This four-component breakdown is the shape that current spec-driven tooling — GitHub Spec-Kit (constitution-first, the natural fit for new modules and microservices extracted from a monolith) and Fission-AI's OpenSpec (change-delta, with its explicit «*built for brownfield, not just greenfield*» positioning) — produces and consumes. Both tools compose with the architecture and framework in this article; the choice between them is whether the project's governing baseline is the system's current behavior or a forward-looking constitution.

Agentic loops are the middle path. An LLM acts as an agent that plans, writes code, runs tests, and iterates inside a loop. Two failure modes recur in practice. First, the agent has no *categorical signal* for which existing element needs correction; it either revisits everything (expensive, slow, regression-prone) or revisits nothing in particular (silent omissions persist). Second, on long-running programmes the agent loses track of its own decisions across sessions; assumptions made on day three quietly contradict assumptions made on day twelve. Even the agent's own chain-of-thought is an unreliable witness to what it actually did. Without an external system of record, the agentic approach drifts.

These three styles are not mutually exclusive. Spec-first work is most commonly executed inside an agentic loop in 2026 tooling, and the architecture and framework in this article assume that combination — and add the external grading and gap-resolution discipline that neither style supplies on its own.

All three styles share the weakness this article addresses. They leave the LLM in control of grading its own work. The transpiler chunk reports «*done*» when the chunk compiles; the spec-first agent reports «*complete*» when the tests it wrote itself pass; the agentic loop reports «*verified*» on its own verification step. None produces a categorical, externally-verifiable signal that the legacy behavior has been preserved or explicitly retired. And none is paired with a spec-driven roadmap that lets the organization see, at any moment, what remains.

The architecture and framework in this article take a different approach. They remove the LLM from the grading loop, replace self-report with categorical structural signals, and bind the engineering work to a spec-driven plan that the team — not the model — declares complete.

The Architecture: Four Pillars

Four concepts turn AI-assisted modernization into an **auditable, measurable** process. Each pillar addresses one failure mode that LLM-driven work routinely exhibits (using the safety-engineering vocabulary of FMEA — IEC 60812).

Image 2 — the four pillars diagram

Pillar	Failure mode addressed	Categorical signal it produces
① Dimensions	Modernization planned on the wrong axis; structural blind spots in the source	Structured source model with explicit relational axes
② Traceability	Target code of unknown provenance; no efficient way to match work to spec	Every target element machine-resolvable back to source + spec + decision
③ Coverage Validation	«N% done» self-assessments; silent omissions	Per-dimension covered / eliminated / unmatched count, externally graded
④ Modernization-Gap Resolution	Errors only surface in production; target cannot be improved after first pass	Iterative generate-or-enhance signal that closes the unmatched count

① Dimensions

A *dimension* is a structural axis along which source architecture elements are related. Dimensions are **discovered per project**, not prescribed, and they determine modernization order, partitioning, and what «*complete*» means.

A useful way to think about a dimension is by the relationship it captures, not by how it is stored. In every real project, six to twelve dimensions matter. The mix is project-dependent:

- For an **Oracle-monolith to Postgres database-per-service** modernization, the dominant dimensions are the data dimension (foreign keys, joinable columns, stored-procedure surface) and the transaction dimension (compensating actions, saga boundaries).
- For a **batch-ETL to streaming lakehouse** modernization, the dominant dimensions are the event dimension (producer → topic → consumer) and the data-quality dimension (schema evolution, late-arriving records, deduplication keys).
- For a **mainframe COBOL** modernization, the dominant dimensions are the JCL / batch-job dimension, the file-layout / copybook dimension, and the business-rule dimension excavated from PERFORM chains.
- For a **post-quantum cryptography** modernization, the dominant dimensions are the algorithm-and-key-store dimension, the certificate-chain dimension, and the protocol-handshake dimension.
- For a **classical web-app** modernization — the article's public reference engagement — the dimensions were code (caller ↔ callee), data (foreign-key), API (client ↔ endpoint), plugin / hook (hook-site ↔ handler), security (principal ↔ resource ↔ permission), and a configuration / feature-flag surface.

The choice of *which* dimensions matter for a project is itself the first modernization decision, and like every other decision in this framework it is recorded explicitly.

② Traceability

Every meaningful target element — function, class, method, model, route, configuration key, CDC connector, saga step — carries two kinds of link:

- a **source link** to the source element(s) that shaped it, with a categorical link type (direct, multi-source, aggregating, inferred, schema-level, new, eliminated);
- a **specification / decision link** to the phase specification, dimension specification, and decision record(s) under which it was produced.

The reason traceability matters has shifted with the technology. In transpiler-style work the links were a reviewer convenience — a way for an SME to retrace which lines of legacy source became which lines of target code. In modern AI-driven flows the reviewer is increasingly *another LLM*, not a human: agentic code review is becoming the norm. The question is no longer «*how fast can a human reviewer cross-check this?*» but «*can the reviewer agent retrieve the source element this code was derived from, the spec that scoped it, and the decision that justified its shape?*» Machine-resolvable traceability is what makes that retrieval cheap. The reviewer agent walks the links, pulls the source, pulls the spec, pulls the decision record, and runs its check. Human review collapses to the exception path on the 10-20% of work the agent grader cannot confirm — not the default path on 100% of it. (Fast human review remains a useful side effect; it is no longer the design goal.)

Traceability is also **not a one-to-one mapping discipline**. The same source element may shape several target elements — a single legacy authentication routine is routinely split into a credential validator, a session issuer, and an audit emitter in the target — and the same target element may aggregate behavior from several non-cohesive sources — a Temporal saga step routinely subsumes what used to be three or four scattered procedural blocks under 2PC. The framework only requires that every target element carry *at least one* source link, and that every source element be in one of three categorical states (covered, eliminated, unmatched). Links are many-to-many; the validator checks containment, not cardinality. The implicit principle is coupling-and-cohesion: source elements that belong together end up linked to the same target element, source elements that did not belong together get redistributed.

The representation is an implementation choice. Inline comments above each target element are the simplest option and the one used in the article's reference engagements. Sidecar index files keep the code clean at the price of extra discipline. Graph databases, commit-level metadata, and specification-side mapping tables are all valid. The only requirement is that the links be programmatically parseable — both by the coverage validator and by the reviewer agent.

③ Coverage Validation

Once every target element carries a traceability link, *completeness* becomes a property the project can measure rather than assert. The measurement is performed by a validator the LLM cannot influence — this is the categorical signal that replaces the model's self-report.

A source-agnostic validator performs four steps:

1. **Source inventory.** Parse every source artifact to capture an exact element list.
2. **Target scanning.** Parse every target artifact for traceability links.
3. **Strict matching.** A source element is *covered* only when at least one target traceability link references that *exact source location*. There is no global-name fallback; a target element linked to one source file does not cover a same-named element in a different source file.
4. **Dimension cross-check.** Verify that dependency edges, entity references, and integration points from the dimension specifications are present in the target.

Each source element falls into exactly one of three categories: **Covered** (target code with at least one traceability link pointing to this source element), **Eliminated** (documented as dead, deprecated, or superseded; not ported), or **Unmatched** (no target equivalent — a gap that must be resolved).

The output is a coverage report **per dimension**. A target that covers 95% of source functions but misses 40% of saga compensating actions, or 60% of CDC tombstone events, is flagged immediately, not after the modernization ships. This replaces single-number coverage metrics with a vector that reflects actual structural risk. It also replaces the model's «*the modernization is 87% complete*» — itself a confirmation-biased self-assessment — with a structural count the team can verify line-by-line. The development lead's exit signal becomes a machine-checkable count, not a sentence written by the model.

④ Modernization-Gap Resolution

Pillar ④ is what makes the architecture a modernization engine rather than a verification harness.

When the coverage validator flags a source element as unmatched or low-confidence, the agent treats that as a signal to **correct or enhance the existing target element**, not only to generate new code. The same cycle applies when a new dimension is discovered, when a deferred decision becomes available, or when a new source ↔ target divergence surfaces during semantic verification.

This breaks the one-shot ceiling. A direct-translation approach cannot revise its output after the first pass without re-running the whole translation. A pure agentic approach has no signal pointing at *which* element to revise. Coverage-driven gap resolution provides exactly that signal: a source element whose current coverage is weak, inconsistent, or contradicted becomes an input to the next gap-resolution cycle regardless of whether target code already exists for it.

The mechanism works for any modernization style, including full architectural redesigns. Suppose the source is a monolith and the target is a set of microservices. The validator reports an unmatched source function. It does *not* instruct the agent to translate those lines. Instead the agent reads the source function to understand its business role; locates the correct target service per the accepted architecture; inspects what the service's responsibilities already cover; either implements the missing responsibility in the target idiom or, if a related target element already exists with weak coverage, enhances or corrects the existing element; and adds a traceability link so the validator treats the source element as accounted for.

Image 3 — the coverage and gap-resolution loop

The generate-or-enhance property is the decisive advantage over transpiler-style work and over purely agentic loops. The framework supports gradual modernization and correction of already-written code, not just addition of new code — which is what makes correct modernization possible at scale, and what eventually lets the team retire the legacy system with confidence.

The Pipeline: Five Phases

The four pillars are operationalized by a five-phase pipeline. Phases 1-3 run once at project inception (though they may be re-entered iteratively as evidence accumulates), Phase 4 executes repeatedly — one modernization phase per working session — and Phase 5 begins once enough of the target is ready to run alongside the source.

Image 4 — the five phases at a glance, with overlaps highlighted

The phases are **not strictly sequential**. Work in Phase 4 routinely triggers short returns to Phase 2 (to accept a deferred decision), Phase 3 (to generate the next phase specification), or Phase 1 (when a new dimension or divergence is discovered). Phase 5 typically overlaps with Phase 4 once enough target is ready to deploy alongside the source.

Phase 1 — Knowledge Extraction and Source Specification

Goal. Produce a complete, evidence-based model of the source system and bind it to the business intent of the modernization. (*Knowledge Extraction* is used here in the sense of ISO/IEC 19506 / KDM, the OMG Architecture-Driven Modernization standard, on the foundation Chikofsky and Cross (1990) laid out for source-side analysis.)

Outputs. Source architecture specifications, one per dimension discovered; a requirements document built on the Mission / Goals / Premises / Constraints breakdown plus a Requirements Traceability Matrix; and a platform-divergence catalog of behavioral differences between source and target platforms, seeded in Phase 1 and grown through every later phase.

A short example of the Mission / Goals / Premises / Constraints breakdown for a single target component — the authentication service in a PHP-to-Python modernization — illustrates the shape:

Mission. Authenticate users against the same credential store the legacy system accepts, with no observable behavior change for already-logged-in users.

Goals. (a) verify against legacy password hashes on first login; (b) silently rewrite to argon2id on the next login; (c) preserve cookie name and session-token shape; (d) emit the same audit-log events.

Premises. Flask-Login for session machinery; argon2-cffi for the modern hash; PostgreSQL for the user table; Redis for the session store.

Constraints. No client-side change; no break of existing single sign-on integrations; ≤ 120 ms p99 verification latency; SHA-1 hashes never persisted again.

The same four-line shape is applied to every component the modernization produces. Goals and Constraints become acceptance criteria; Premises become entry-gate dependencies on decision records; Mission becomes the SME-review hook. The breakdown is the shape that current spec-driven tooling expects; it is also the shape that survives translation between brownfield (delta) and greenfield (constitution-first) tools without rewrite.

The governance container for the requirements document is itself a Phase-1 decision. An Agile project may use a *project charter*; a Waterfall project may use a *project definition plan* with a work-breakdown structure; a regulated programme may use a *programme vision*. The framework mandates the four components, not the name of the container.

Phase 2 — Architect, Product, and SME Decisions

Goal. Propose every non-trivial target-architecture and modernization-strategy choice; accept each decision with the stakeholders responsible for the outcome; record the rationale so the decision is auditable and re-openable.

Each decision record contains: context, considered options, trade-off analysis, decision, consequences, and confirmation criteria — the MADR-style Architecture Decision Record originated in Nygard (2011) and now widely adopted in the software-architecture community. The LLM proposes options; humans accept them. This is the second place where the framework refuses to let the LLM grade its own work: the model never confirms a decision unilaterally.

A decision accepted by the implementer alone is a design note. A decision accepted by named stakeholders — product team, project architect, subject-matter experts, security or operations leads — is a governance artifact, and is the product owner's primary mechanism for re-opening a trade-off when business needs change.

Decisions may be accepted in deferred mode. P0 decisions (flow variant, target stack, primary data engine) are normally accepted in Phase 2 itself. P1 decisions (ORM, auth, saga substrate, background-worker fabric) can remain *proposed* until the modernization phase that first needs them. P2 decisions (logging, observability, internationalisation) can evolve in parallel with implementation. Decisions in the reference engagements were partitioned P0/P1/P2 by acceptance timing; examples appear in the walkthrough below.

Phase 3 — Target Specifications

Goal. Produce a single set of specifications that covers both the target architecture and the modernization strategy for each phase. The framework does not separate «*target architecture specs*» from «*modernization plan specs*» — they are the same set of documents, describing the same phase from two angles. This unification is what eliminates the drift that accumulates when architecture and strategy plans are maintained separately.

Each modernization phase is described by three artifacts: a **specification** (user-visible behavior, acceptance criteria, success criteria, explicit scope and anti-scope); a **plan** (technical context, decision-record dependencies, modernization batches in dependency order, risk assessment, entry and exit gates); and a **task list** (actionable steps, parallel markers, source-element cross-references).

Generation is iterative, not big-bang. The team generates the spec set for the first unblocked phase (often a walking skeleton or foundational slice), executes it through Phase 4, and only then generates the next phase's specs. At that point, decisions that were previously deferred may become needed; that triggers a short loop back to Phase 2 to accept them.

Phase 4 — Modernization

Goal. Turn each phase specification into verified target artifacts. Phase 4 runs once per modernization phase, in a fresh working session so that context is clean and the session handoff artifact is the only carrier of cross-phase state.

 *Image 5 — the Phase 4 inner loop*

Per batch in the modernization phase:

1. Produce target artifacts with traceability links.
2. Run structural coverage analysis per dimension → produce a gap report.
3. Resolve gaps: generate new target code OR enhance / correct existing target code.
4. Run semantic verification on high-risk elements, against the platform-divergence catalog (line-by-line with raw source quoting).
5. Run build / static analysis / unit tests.
6. Run integration and end-to-end tests at the current scope.
7. If anything fails, narrow scope and return to step 1.

The phase exit gate is a checklist, not a narrative: zero unmatched in-scope elements, all automated tests green, all decision records the phase depended on in *accepted* status, new platform-divergence catalog entries merged, session handoff note written. The exit gate is the only place where the team — not the model — declares the phase done. The development lead's per-dimension coverage report replaces percentage-of-LOC metrics; modernization can stop mid-phase without losing audit trail.

A phase may emit artifacts into any number of target repositories. Single-repository, one-to-many, and many-to-many outputs are all natural results of the same loop.

Phase 5 — Hybrid Deployment and Cutover

Goal. Run the target alongside the source under realistic conditions; shift traffic progressively; decommission the source system when every responsibility is covered.

A *big-bang cutover* — a single-event, system-wide replacement in which the legacy and modernized systems are swapped in one coordinated release, with no parallel-run or incremental traffic-routing phase (cf. Fowler 2021; O'Leary 2000) — is prohibitively risky for most systems. The pipeline treats the coexistence architecture itself as a decision, not a default. Five release modes are commonly used:

Mode	When to use
Big-bang single cutover	Small / mid-scale projects; full regression coverage feasible
Gradual coexistence	Large systems where full cutover risk is unacceptable
Incremental traffic shift	Systems with a gateway, service mesh, or feature-flag layer
Read-first / write-later	Data-heavy systems where writes are the highest risk
Parallel-run comparison	Pipelines, reports, or computations where output correctness is observable

The operations lead inherits the full deployment topology — infrastructure-as-code modules, service-mesh and gateway configuration, data-migration scripts, observability configuration — together with a defensible decommissioning gate. Because every legacy element is in one of three categorical states and the unmatched count is the exit gate of every preceding phase, the team enters Phase 5 with an unambiguous answer to the question every steering committee asks: «*what would we lose if we turned off the source system tomorrow?*» The answer is the unmatched list, and at exit-gate completion that list is empty by construction.

The security / compliance lead inherits the corresponding security guarantee: every credential, every cipher, every security-relevant code path is traceable to the source element it derives from and to the decision that justified its target form.

The Architecture and Framework in Practice

The architecture and framework were validated on two engagements with materially different shapes. The first is a memory-safety-critical C-to-Rust modernization of a native system, where the dominant dimensions were data flow, ownership, and the integration surface between unsafe and safe code regions. The second — the public reference engagement — is a complete *PHP-to-Python* rewrite of a non-trivial self-hosted web application, where the dominant dimensions were the plugin / hook surface, the security model, and the API surface. The same loop, the same coverage validator, the same gap-resolution discipline carried both engagements through to retirement of the legacy system. The C engagement exercised semantic-verification regions the web engagement did not; the web engagement exercised plugin and authentication regions the C engagement did not.

Image 6 — artifact flow diagram

End-to-end validation across both engagements has been at the mid-tens-of-thousands-of-lines scale. Claims of fit at full enterprise scale rest on architectural compatibility with the at-scale patterns of Saavedra et al. (2025) and the TRACTOR performers, not on direct evidence at that scale.

In the web engagement, Phase 1 produced one source architecture specification per dimension discovered, plus the platform-divergence taxonomy. Phase 2 produced a small set of P0 decisions on flow variant, target framework, and primary database engine, accepted up front; the remaining P1 decisions on ORM strategy, session management, password-hash migration, feed-credential encryption, plugin substrate, background-worker fabric, feed parser, and HTTP client were accepted just-in-time before the phases that needed them. Phase 4 ran the modernization loop across the planned phases. Each function, class, model, and route carries inline traceability comments of the form:

```
1 # Source: ttrss/include/functions.php:authenticate_user (lines 706-771)
2 # Specs:  source-security-model, foundation-phase
3 # Decisions: password-migration
4 def authenticate_user(login: str, password: str) -> Optional[User]:
5     ...
```


The coverage validator was implemented as a Python script that parses both source (with tree-sitter for exact element boundaries) and target traceability comments, producing a per-dimension report. The platform-divergence catalog grew through every phase into a checklist-driven audit with falsifiable entries, transforming what would otherwise have been ad-hoc semantic review.

A representative decision-record excerpt:

```
1  # Select Python Web Framework
2
3  ## Considered Options
4
5  1. Flask — closest match to source handler dispatch; native session and CSRF
6  2. FastAPI — modern async; no built-in sessions; client-side changes for CSRF
7  3. Django — full-featured but over-engineered for this codebase
8
9  ## Decision: Flask
10
11  Rationale: minimizes behavioral-contract divergence; preserves the client-server protocol; lowest
12  decision cost given the accepted authentication and session decisions.
13
14  ## Consequences
15
16  - zero client-side changes required
17  - native CSRF and session model identical to source
18  - synchronous runtime; async deferred to Celery workers
```

The modernization shipped with a target that explicitly documents which source elements were eliminated (with the decision record that justified each elimination), and a deployment topology — single-host Docker Compose stack appropriate for the reference workload, with the topology mapping to Kubernetes or ECS at production scale documented as an out-of-scope follow-on — backed by pgloader-based data-migration scripts and a staged cutover plan.

The pattern is consistent with what the broader research community has reported on AI-driven modernization at larger scale. Google's at-scale work (Saavedra et al. 2025) describes 39 modernization programmes across 93,574 source edits, with the same generate-validate-correct shape as Phase 4, although without per-dimension coverage or unmatched-element gap resolution as first-class signals. DARPA's TRACTOR performers (verified lifting with LLMs, skeleton-guided translation, oracle-validated transpilation) address correctness of individual translations and are complementary to — not instances of — Pillar ©'s system-level completeness mechanism.

Looking Ahead

Five developments in the next 18-24 months will materially affect this architecture and framework:

- **More mature source-inventory tooling.** Tree-sitter-based parsers are already in use in the reference engagements; the next round of progress will come from richer language-server indexes and code-intelligence graph stores (SCIP, stack-graphs, CodeQL) that make the source side of the coverage validator faster and more precise on enterprise codebases.
- **Living divergence catalogs across organizations.** Today each project grows its own platform-divergence catalog. Shared, versioned catalogs for common platform pairs (Java 8 → 21/25, .NET Framework → .NET 9, Oracle PL/SQL → PostgreSQL, AngularJS → modern SPA, 2PC → Temporal, batch ETL → streaming, classical → post-quantum crypto) would amortize the cost across many programmes.
- **Typed orchestration frameworks.** Substrates such as Microsoft Agent Framework are converging on declarative LLM-agent coordination. The five-phase pipeline maps naturally onto them.
- **Formal verification for the semantic-verification step.** For safety-critical sub-systems — exactly the kind of work the C-to-Rust engagement exercised — semantic verification can be lifted from checklist-driven audit to formal

equivalence proofs, with VerMCTS, Clover, and automatic Dafny annotation as the current research frontier.

- **Confirmation-bias-aware evaluation.** The next generation of benchmarks will measure work the agent has not seen at training time and will refuse to accept the agent's self-report as a completion signal. The architecture and framework here are an early instantiation of that principle applied to enterprise modernization.

Feasibility of AI-assisted modernization at non-trivial scale is no longer in serious doubt. The remaining question is which architecture and framework produce evidence the auditor, the product owner, and the operations lead each verify independently, and that establishes an unambiguous decommissioning point for the legacy system. This article describes one practical answer: dimension-driven, traceability-first, coverage-validated, gap-resolving rather than translating in one shot, and keeping the LLM out of the grading loop at every checkpoint.

References

Foundational frameworks and patterns

- Chikofsky, E. J., Cross, J. H. (1990). *Reverse Engineering and Design Recovery: A Taxonomy*. IEEE Software 7(1). The foundational vocabulary for source-side analysis underpinning Phase 1.
- ISO/IEC 19506:2012. *Architecture-Driven Modernisation: Knowledge Discovery Meta-Model (KDM)*. The international standard underpinning Phase 1's *Knowledge Extraction* terminology.
- Nygard, M. (2011). *Documenting Architecture Decisions*. The ADR convention; MADR project at adr.github.io/madr.
- O'Leary, D. E. (2000). *Enterprise Resource Planning Systems*, Chapter 11 (*Big Bang versus Phased*). Cambridge University Press. Canonical definition of big-bang cutover.
- Fowler, M. (2021). *Patterns of Legacy Displacement*. The Strangler Fig / big-bang contrast and the patterns underpinning Phase 5.
- IEC 60812. *Failure Mode and Effects Analysis (FMEA)*. The provenance of the *failure mode* vocabulary used throughout this article.

LLM confirmation bias and reward hacking (current generation)

- Mitropoulos, D., et al. (March 2026). *Measuring and Exploiting Confirmation Bias in LLM-Assisted Security Code Review*. arXiv preprint. The 16-93% detection-rate drop under «bug-free» framing; 88% adversarial-PR success against Claude Code under iterative project configurations.
- Anthropic (February 2026). *Claude Opus 4.6 System Card* at anthropic.com. Formal cataloging of «misrepresenting tool results» and «premature claims of success» as named failure modes.
- Anthropic (April 2026). *Claude Code Postmortem*. Disclosure of silent regressions in March-April 2026 that caused premature task-completion declarations.
- arXiv:2605.02964 (May 2026). *Reward Hacking Benchmark*. Thirteen current-generation models tested; 72% of reward-hacking episodes include explicit chain-of-thought rationale.
- arXiv:2604.13602 (April 2026). *Reward Hacking in the Era of Large Models*. The Proxy Compression Hypothesis: reward hacking as a structural property of RLHF / RLVR alignment, not a prompt-fix problem.

AI-assisted modernization at scale

- Saavedra, N., et al. (Google, 2025). *Migrating Code At Scale With LLMs At Google*. arXiv:2504.09691. Thirty-nine programmes; 93,574 source edits.
 - DARPA. *Translating All C to Rust (TRACTOR)*. Programme page at darpa.mil. MIT Lincoln Laboratory test and evaluation at ll.mit.edu/r-d/projects/translating-all-c-rust-tractor-benchmarks.
 - Capgemini. *AI-Assisted Legacy Modernization (CAALM)*, 2025. The Capgemini methodology underpinning the practice in which this architecture and framework were developed.
-

About the author. Dmytro Rudakov developed and validated the architecture and framework described here within Capgemini's AI-Assisted Legacy Modernization (CAALM) practice, across two genuinely different engagements: a memory-safety-critical C-to-Rust modernization of a native system, and a complete PHP-to-Python rewrite of a non-trivial self-hosted web application. The reference engagement artifacts — source architecture specifications, decision records, phase specifications, the platform-divergence catalog, and the full target codebase — are available as a working example.